OpenZeppelin | security

Volatile Pair Hook Audit



Uniswap

October 9, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents 2

Summary 3

Scope 4

System Overview 5

Security Model and Trust Assumptions 6

Low Severity 10

L-01 Free Price Moves Through Empty Liquidity Are Recorded as Price Impact 10

L-02 Per-Block Fee Windows Are Deployed to Chains With Block Times From 12 Seconds to 250 Milliseconds 11

L-03 Capture Term Charges Counter-Trend Users During Informed Moves 11

Notes & Additional Information 12

N-01 initializePool Does Not Emit FeeConfigUpdated 12

N-02 IVolatileFeeConfiguration Does Not Declare the feeConfig and feeState Getters 12

N-03 Odd Resting Spread Is Split Unevenly at Reset 13

N-04 Variable Initialized With Its Default Value 13

N-05 Redundant Return Statement 13

Conclusion 15

Appendix 16

Issue Classification 16

Summary

Type	DeFi
Timeline	From 2026-10-05 To 2026-10-08
Languages	Solidity

Total Issues	8 (0 resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	0 (0 resolved)
Medium Severity Issues	0 (0 resolved)
Low Severity Issues	3 (0 resolved)
Notes & Additional Information	5 (0 resolved)
Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the Uniswap/v4-hooks-internal repository at commit 673e227c.

In scope were the following files:

```
src
    └── volatile
        ├── base
        │   └── VolatileFeeConfiguration.sol
        ├── interfaces
        │   ├── IVolatileFeeConfiguration.sol
        │   └── IVolatilePairHook.sol
        ├── libraries
        │   ├── RelativeMoveLibrary.sol
        │   └── VolatileFeeCalculation.sol
        └── VolatilePairHook.sol
```

System Overview

The `VolatilePairHook` is a Uniswap v4 hook that sets a separate dynamic LP fee for each swap direction on pools of volatile pairs, such as ETH/USDC. It aims to lower the loss-versus-rebalancing (LVR) that liquidity providers suffer after price moves. It also returns part of the backrun profit to liquidity providers and cuts the profit of the back leg of a sandwich. In blocks where the price does not move, the quoted spread stays constant. A swap that buys `token0` pays the ask fee, and a swap that sells `token0` pays the bid fee.

Fee Model: Each fee has three parts. The permanent fees of the two sides always add up to a configured resting spread. After a price move, the side that the price moved toward (the leaning side) becomes more expensive, and the other side becomes cheaper by the same amount. This lean carries over to later blocks, within a configured band. For one block after a move, the side that trades against the move (the backrun side) also pays a temporary fee. This fee has a restore piece and a capture piece, each gated by its own price-impact threshold. The restore piece adds back the drop in the permanent fee of the backrun side. The capture piece charges a share of the move. Finally, every swap pays a mid-block extra that grows with the distance between the current price and the price at the first swap of the block (the block open). The `VolatileFeeCalculation` and `RelativeMoveLibrary` libraries measure each move as an exact ratio in price space and round each fee term up, in favor of liquidity providers. Every fee is clamped to `MAX_FEE_E6`. This bound keeps the combined LP and protocol fee below 100% for any allowed protocol fee.

Swap Flow: The `PoolManager` calls `beforeSwap` on every swap. On the first swap of a new block, the hook compares the current price with the stored open of the previous block. It then steps the permanent fees, prices the temporary fee, and stores the current price as the new block open. Each swap then pays the starting fee of its side plus the mid-block extra, and the hook returns this fee with the v4 override flag. The `getFee` view function runs the same computation without writing state. The fee never depends on the swap amount.

Upgradeable Hook Architecture: The registered hook is an ERC-1967 proxy that delegates to a UUPS implementation of `VolatilePairHook`. The proxy address is mined to encode the hook permission flags, so these flags are fixed for the life of the hook. The shared `BaseUUPSHook` and `BaseDynamicFeeHook` contracts, which `StablePairHook` also inherits, define these flags, the access control roles, and the upgrade checks. Only `beforeInitialize` and `beforeSwap` perform work. The add-liquidity and after-swap

callbacks are inert, and the remove-liquidity, donate, and return-delta flags are disabled. Pools can only be created through `initializePool`. This function checks the pool key, stores a validated fee configuration, and initializes the pool on the `PoolManager` in one call. The `VolatileFeeConfiguration` contract holds the per-pool configuration and fee state in ERC-7201 namespaced storage and validates every configuration write.

Security Model and Trust Assumptions

The hook defines three privileged roles through OpenZeppelin `AccessControl`. They are granted when the proxy is initialized. Other actors interact with the pools without on-chain roles, but their behavior shapes the economic outcome of the fee mechanism.

Admin (`DEFAULT_ADMIN_ROLE`): The admin grants and revokes every role, including its own, and authorizes implementation upgrades. The `initialize` function rejects the zero address and the hook itself as the admin. On upgrade, the hook only checks that the new implementation uses the same `PoolManager` and the same hook permission flags. It does not check storage layout compatibility. The admin is therefore trusted to keep the namespaced storage append-only across upgrades. An upgraded implementation can replace the fee logic of every pool and charge any LP fee that v4 accepts. It can also revert in the swap or add-liquidity callbacks to halt swaps and new deposits. The `afterInitialize`, `beforeAddLiquidity`, `afterAddLiquidity`, and `afterSwap` flags are enabled but inert. They are kept as room for future upgrades, because the flags cannot change after deployment. The remove-liquidity, donate, and return-delta flags are disabled at the proxy address. As a result, no implementation can block liquidity removal or return deltas that change swap amounts. The admin handover is a single step. If the last admin renounces its role, upgrades and role management are frozen permanently. The admin is trusted to be a secure account, to grant the operational roles only to trusted addresses, and to review every implementation before it authorizes the upgrade.

Pool Initializer (`POOL_INITIALIZER_ROLE`): The pool initializer creates pools through `initializePool`. It chooses the pool key, the initial price, and the initial fee configuration. The function requires the dynamic fee flag and this hook in the pool key. Direct initialization on the `PoolManager` reverts in `beforeInitialize`. As a result, every pool that uses the hook has a validated configuration, and no configuration exists without a pool. The pool initializer is

trusted to create pools only for pairs that the algorithm is designed for, and with an initial price that reflects the market.

Config Manager (`CONFIG_MANAGER_ROLE`): The config manager can update the configuration of any initialized pool, alone or in a batch, at any time and without delay. Validation bounds the parameters as follows:

- The resting spread is nonzero and at most twice `MAX_FEE_E6`.
- The maximum permanent fee of a side lies between half the resting spread and the full resting spread, and never exceeds `MAX_FEE_E6`.
- The per-block lean step is at most the width of the band in which the permanent fees can move.
- The capture and restore fractions are at most 100%, and the capture and restore thresholds are below 100%.
- The skew rate and the mid-block growth rate are not capped. Their effect is limited by the maximum lean step, the cap on the mid-block extra, and the `MAX_FEE_E6` clamp.

The accepted range still lets the fee of either side reach `MAX_FEE_E6`, just below 100%. If the maximum permanent fee equals the resting spread, a strong enough trend drives the permanent fee of the other side to zero. The temporary fee and the mid-block extra still apply on top. The restore settings have no defaults. A restore fraction of zero disables the restore piece, while a full fraction with a zero threshold adds back the whole permanent fee drop on every move. Every write also resets the fee state of the pool immediately, even within a block. The reset discards the accumulated skew, the temporary fee, and the block open, so the next swap starts from the current pool price. If an update lands after a price push and before its correction, the correction pays only the neutral permanent fee instead of the temporary fee. Every window of the fee mechanism lasts one block. The same parameters therefore behave differently on chains with different block times. The config manager is trusted to set economically sound parameters for each pair and each chain, and to set the restore settings on purpose. It is also trusted to time its updates so that a reset does not hand a discount to a pending correction. Because an update can change the fee between a quote and its execution, swappers are expected to protect their swaps with slippage limits.

Swappers, Arbitrageurs, and Block Builders: These actors are not on-chain roles. Any address can swap, and the first swap of each block advances the fee state and sets the block open, whatever its size. Whoever orders the transactions in a block therefore decides which

price the hook records as the open. The fee mechanism is designed to reduce LVR and backrun or sandwich profit, not to remove them. The following behaviors are accepted:

- The temporary fee applies to every swap on the backrun side for the whole block, not only to the first one.
- If an unrelated swap lands in the block after a price push, a backrun that waits for a later block avoids the temporary fee. If no swap lands in the following block, a backrun in the block after it still pays the temporary fee.
- A backrun split into many swaps within one block pays about half the mid-block extra of a single large swap.
- Each move is measured relative to the older price. An upward move has no upper bound, while a downward move always stays below 100%. A move up and back to the same price therefore leans the permanent fees slightly toward the ask.
- Each lean step rounds up, so any nonzero move steps the permanent fee by at least one pip. On a quiet pool, a tiny swap in every block can walk the permanent fee of one side down. The saving is at most half the resting spread per unit traded at the lowered fee, and slippage and gas are expected to make this unprofitable.
- A swap through a price range without active liquidity moves the price at no cost. This swap can set the block open, the skew, and the temporary fee.

Liquidity providers are assumed to accept these limits as part of the fee policy.

Liquidity Providers: Liquidity providers are not on-chain roles. They can add and remove liquidity without restriction. Every fee that the hook sets is an LP fee and ignores the protocol fee of the pool. If the protocol fee is enabled, v4 charges it on top of the LP fee. `MAX_FEE_E6` is set to 999,998 pips so that the LP fee compounded with any allowed protocol fee stays below 100%. Each fee term rounds up on its own, and the move measurement can only overstate a move. A fee can therefore exceed the exact value by a few pips, in favor of liquidity providers. The admin and the config manager set the fees that determine the revenue and the exposure of liquidity providers. Liquidity providers rely on these roles to keep the fee policy aligned with their interests.

Integrators and Quoters: The `getFee` function returns the fee that the next swap in each direction would pay at the current state. The fee never depends on the swap amount. This value is not fixed for the block. Later swaps in the same block can pay more as the price moves away from the block open. Integrators are expected to quote close to execution and to treat `getFee` as an estimate for the next swap only.

Deployer: The deployer must deploy the proxy with the `initialize` call in its constructor. Otherwise, the first caller can initialize the proxy and take the admin role. The `ERC1967Proxy`

version in use reverts when it is deployed with empty initialization data. The deployer must also mine a proxy address that encodes exactly the hook permission flags, because these flags cannot change afterward. The deployment script in the repository performs both steps in one transaction. If the proxy already exists, the script checks its implementation and its roles instead.

External Dependencies: The hook relies on the `PoolManager` skipping hook callbacks when the hook itself initializes a pool. This behavior makes `initializePool` the only way to create a pool. The hook reads the current price from the `PoolManager` and the block number from `BlockNumberish`. On chains that expose the Arbitrum `ArbSys` precompile, `BlockNumberish` returns the L2 block number. Elsewhere, it returns `block.number`. The hook treats every swap whose block number is not greater than the stored one as part of the same block. The `openzeppelin-contracts` dependency is pinned at commit `45f032d1`, between v5.6.1 and v5.7.0, and `openzeppelin-contracts-upgradeable` is pinned at v5.5.0. The `UUPSUpgradeable`, `AccessControl`, and `ERC1967Utils` files that the hook uses do not differ in behavior between these versions.

Low Severity

L-01 Free Price Moves Through Empty Liquidity Are Recorded as Price Impact

The hook reads only the pool's sqrt price before each swap and never reads the active liquidity. Its `afterSwap` callback is inert. In Uniswap v4, a swap that starts at a price with no active liquidity consumes nothing and pays no fee. It moves the price to the next initialized tick or to the caller's limit. The hook records such a move as a paid one. It uses the move as the block open for the displacement extra. It also uses the move as the reference for the next block's lean step and temporary fee. Item 8 of the accepted-behaviors document acknowledges that a free move can set the fee state and limits the concern to thin or brand-new pools. However, the condition holds in any pool whose liquidity sits entirely on one side of the price. This includes a token launch pool for as long as it has not been bought through. It also includes any pool whose price has moved beyond every position.

If the first swap of a block moves freely away from the liquidity, every subsequent trade into liquidity in that block backruns the recorded displacement. The hook charges these trades the capture of a move that nobody paid for. A free return as the first swap of the next block records the previous move as block-to-block price impact and arms the temporary fee on the same side. Alternating the two, one dust swap per block, keeps the fee on the side that trades into liquidity far above any ordinary slippage tolerance. This happens in every block, so legitimate trades revert for as long as the attacker pays gas. On the L2s in the deployment parameters, this requires one transaction every fraction of a second.

Consider documenting in item 8 that the condition is not limited to thin or brand-new pools. Any pool whose liquidity lies entirely on one side of the price can have fees for trades into liquidity kept above ordinary slippage tolerances. This costs one dust swap per block. Consider also documenting that the configuration manager's reset does not help because the state can be rebuilt in the next block. Consider also stating that the hook is intended for pools kept two-sided, so that pool creation and liquidity management follow that policy.

L-02 Per-Block Fee Windows Are Deployed to Chains With Block Times From 12 Seconds to 250 Milliseconds

Every window of the fee design is one block: the interval over which price impact is measured, the lifetime of the temporary fee and the cap on the lean step. The displacement window of the mid-block extra is also one block, with the block number read through `BlockNumberish`. The specification names Uniswap v4 mainnet as the target and never mentions block time. However, the deployment parameters span chains with block times from 12 s to 250 ms.

As a result, the same configuration behaves differently on each chain. Per-block price moves scale with the square root of block time. Therefore, a capture threshold and a step cap tuned for one chain are several times off on another. A move that is one block of impact on mainnet arrives on a sub-second chain as many moves below the threshold. These moves incur no capture while the lean still saturates within seconds. On such chains, a block rarely holds two swaps of the same pool, so the mid-block extra rarely applies, and the one-block windows last a fraction of a second.

Consider documenting that every window is measured in blocks and scales with the block time of the target chain. Include the effects on the threshold, the step cap, the temporary fee and the mid-block extra.

L-03 Capture Term Charges Counter-Trend Users During Informed Moves

The design describes the temporary fee and the mid-block extra as taking part of the profit when a large uninformed trade moves the price and someone backruns it. The code, however, arms the temporary fee on every block whose open-to-open move exceeds the threshold, regardless of why the price moved. It likewise charges the mid-block extra to every swap against the block's displacement. Item 1 of the accepted-behaviors document notes that every swap on that side pays it.

During a sustained move tracked by arbitrageurs, every block's move is informed, and there is no backrun profit to share. The backrun side is the ordinary flow trading against the trend. Those users pay the capture of each block's move on top of the lean for the whole duration of the move. This fee is a multiple of the neutral fee. They pay more still when the move reverses, while flow in the direction of the move pays at most the full spread. Once the move ends, the lean stays at its cap until cumulative opposite moves of about half the spread accrue, so the

asymmetry outlives the volatility. The documents explain how the fee is computed in every case but justify the capture term only for the uninformed-push case.

Consider documenting the fee a user experiences during sustained moves. In particular, document that the capture term is charged in every block of an informed trend and falls on counter-trend flow. Also document that the lean persists after the move. Consider also validating the design in that regime with a simulation study such as the one maintained for the stable hook. The claim that the lean lowers LVR depends on how counter-trend flow responds to the fee.

Notes & Additional Information

N-01 `initializePool` Does Not Emit `FeeConfigUpdated`

The `initializePool` function of the `VolatilePairHook` contract seeds a pool's fee configuration through `_updateFeeConfig`. It emits `PoolInitialized`, which carries the configuration, but does not emit `FeeConfigUpdated`. The two configuration-manager entry points, `updateFeeConfig` and `batchUpdateFeeConfig`, emit `FeeConfigUpdated` for the same write. Indexers that track a pool's configuration must therefore handle two different events for the same state change.

Consider emitting `FeeConfigUpdated` from `_updateFeeConfig` so that every configuration write produces the same event.

N-02 `IVolatileFeeConfiguration` Does Not Declare the `feeConfig` and `feeState` Getters

The `VolatileFeeConfiguration` contract defines the `feeConfig` and `feeState` view functions, but the `IVolatileFeeConfiguration` interface does not declare them. Integrators that code against the interface cannot read a pool's configuration or fee state without importing the implementation.

Consider declaring both getters in `IVolatileFeeConfiguration`.

N-03 Odd Resting Spread Is Split Unevenly at Reset

Every configuration write resets the ask permanent fee to `restingSpreadE6 / 2` and derives the bid permanent fee as the remainder. For an odd spread, the ask receives the floor and the bid the ceiling. The neutral state therefore quotes the bid one pip above the ask. The documented reset value of `restingSpreadE6 / 2` does not hold for the bid. Only the role-gated configuration writes can reach this state. The fixed total keeps a round trip at exactly the spread. The lean band stops the pip from compounding over any sequence of moves. The unchecked subtraction in `temporaryBackrunFee` relies on the reset bid not exceeding `maxSkewE6`, and the `MaxSkewTooSmall` check protects it. The effect is therefore cosmetic.

Consider requiring `restingSpreadE6` to be even in `_validateRestingSpreadE6` so that the neutral state is symmetric. If odd spreads are to remain allowed, consider documenting which side receives the extra pip.

N-04 Variable Initialized With Its Default Value

Within `VolatileFeeConfiguration.sol`, the `i` variable is initialized with its default value. With the compiler version and optimizer settings that the project uses, the explicit initialization produces the same bytecode as leaving the variable uninitialized, so there is no cost difference.

Consider not initializing variables with their default values to improve code quality and consistency.

N-05 Redundant Return Statement

Throughout the codebase, there are multiple instances of redundant return statements:

- The `return (IHooks.beforeSwap.selector, BeforeSwapDeltaLibrary.ZERO_DELTA, feeE6 | LPFeeLibrary.OVERRIDE_FEE_FLAG);` statement in `VolatilePairHook.sol`.
- The `return (config.restingSpreadE6, ... , config.restoreThresholdE6);` statement in `VolatileFeeConfiguration.sol`.

- The `return (state.blockNumber, state.sqrtPriceTopX96, state.askPermFeeE6, state.tempFeeE6, state.tempOnBid);` statement in `VolatileFeeConfiguration.sol`.
- The `return RelativeMoveLibrary.pack(0, FixedPoint96.Q96, false);` statement in `VolatileFeeCalculation.sol`.
- The `return RelativeMoveLibrary.pack(FixedPoint96.Q96 - ratioX96, increased ? ratioX96 : FixedPoint96.Q96, increased);` statement in `VolatileFeeCalculation.sol`.

Furthermore, two functions mix both styles, assigning their named returns on one path and returning explicitly on the other:

- The `return (poolState, config, currentSqrtPriceX96, false);` statement in `VolatilePairHook.sol`, where `isNewBlock` is assigned on the other path.
- The `return permFeeE6;` statement in `VolatileFeeCalculation.sol`, where `baselineFeeE6` is assigned on the other path.

Consider removing the redundant return statements in functions with named returns, and using a single style within each function, to improve the readability of the codebase.

Conclusion

The `VolatilePairHook` sets a separate dynamic LP fee for each swap direction on Uniswap v4 pools of volatile pairs. It leans the permanent fees toward the last price move and charges a one-block temporary fee on the backrun side. Within a block, it raises the fee as the price moves away from the block open. Its purpose is to lower LVR for liquidity providers and to return part of the backrun profit to them. OpenZeppelin audited the hook, its fee calculation libraries, and its configuration contract.

Three low-severity issues and five notes were identified. No issue concerns the arithmetic of the fee calculation. The configuration validation bounds the parameters on which its unchecked operations depend. The low-severity issues concern the economic behavior of the fee mechanism under conditions that the documentation does not fully cover. One shows that a swap through a price range without active liquidity sets the fee state at no cost. If all liquidity of a pool lies on one side of the price, one dust swap per block can keep the fee for trades into it above ordinary slippage limits. Another shows that every fee window lasts one block, while the deployment parameters cover chains with block times from 12 seconds to 250 milliseconds. A configuration tuned for one chain can therefore behave differently on another. The last shows that, during a sustained informed move, the capture term charges counter-trend users in every block, and the lean persists after the move ends. The documentation justifies this term only for backruns of uninformed trades. The notes concern event consistency at pool creation, getters missing from an interface, the uneven split of an odd resting spread at reset, and code style.

The technical specification states the algorithm, its invariants, and its rounding direction. A separate document lists the behaviors that the Uniswap team considered and accepted. The test suite includes unit, fuzz, invariant, and vector-based tests for the fee calculation. The outcome for liquidity providers depends on the parameters chosen for each pair and chain, and on the conduct of the privileged roles. Documenting the fee that users pay during sustained moves and across block times would support these choices. A simulation study, such as the one maintained for the stable hook, would also help validate the parameters before deployment.

OpenZeppelin thanks the Uniswap team for its cooperation throughout the audit and for the detailed technical documentation provided with the codebase.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.